

Final Laboratory Report: Secure Hybrid Cryptographic Pipeline Implementation

Matthew Love

May 4, 2026

Cryptography and Network Security - CSC-4575-001

1. Executive Summary

In the modern threat landscape, the "Key Distribution Problem" remains one of the most significant hurdles in network security. While symmetric ciphers offer high-performance encryption for bulk data, **the secure exchange of secret keys over untrusted media** requires a sophisticated, multi-layered approach. This project marks the transition from theoretical cryptographic primitives to practical engineering by implementing a secure hybrid pipeline. By **layering asymmetric key exchange, digital signatures, and authenticated symmetric encryption**, we "bootstrap" a secure session that provides a defense-in-depth posture against both passive eavesdropping and active interception.

The three-week engineering project resulted in a functional pipeline between two Docker nodes (Alice and Bob) using the following cryptography methods/concepts:

- AES-256-GCM (Galois/Counter Mode) for high-speed encryption with built-in integrity.
- X25519 (Elliptic Curve Diffie-Hellman) to negotiate a shared secret session key.
- RSA-2048 with PSS padding and SHA-256 to ensure the authenticity of the handshake.

The final architecture satisfies the following foundational security goals:

- **Confidentiality: Achieved via AES-256 encryption.** By ensuring the ciphertext provides no advantage to a computationally bounded attacker, we protect sensitive "CLASSIFIED" payloads from unauthorized disclosure.
- **Integrity: Achieved via the GCM Authentication Tag.** GCM performs "Authenticated Encryption," allowing the system to detect tampering (such as bit-flipping) at the hardware level, preventing attackers from altering message contents.
- **Authenticity: Achieved via RSA-2048 digital signatures using Probabilistic Signature Scheme (PSS) padding.** This binds the ephemeral session keys to a verified long-term identity, thwarting Man-in-the-Middle (MitM) attacks.
- **Forward Secrecy: Achieved through the use of ephemeral X25519 keys.** Because session keys are never stored and are independent of the long-term RSA keys, a future compromise of Bob's RSA private key will not allow an attacker to decrypt past recorded sessions.

These effectiveness of these cryptography methods/concepts rely on following the appropriate sequence of operations to prevent identity substitution.

[Project Highlights and Takeaways Video](#)

2. Protocol Architecture and Authenticated Handshake Flow

The sequence of operations in a cryptographic handshake is as important as the algorithms themselves. If a system transmits encrypted data before the receiver's identity is verified, it remains vulnerable to impersonation. The protocol implemented here ensures that identity verification occurs as a required part of key negotiation.

Sequence Breakdown

The Week 3 authenticated handshake follows this sequential flow (between the Alice (client) and Bob (server) Docker nodes):

1. **Alice:** generate ephemeral X25519 private key a and public key A (32 bytes).
2. **Alice:** sign A with her long-term RSA-2048 private key: $\text{Sig}_A = \text{Sign}_{sk_A}(A)$ (PSS+SHA256, ~256 bytes).
3. Alice $\rightarrow$ Bob: send $\text{Sig}_A || A$.
4. **Bob:** receive $\text{Sig}_A || A$; verify with $\text{Verify}_{pk_A}(\text{Sig}_A, A)$. Abort on failure.
5. **Bob:** generate ephemeral X25519 private key b and public key B (32 bytes).
6. **Bob:** sign B with his long-term RSA-2048 private key: $\text{Sig}_B = \text{Sign}_{sk_B}(B)$.
7. **Bob $\rightarrow$ Alice:** send $\text{Sig}_B || B$.
8. **Alice:** receive $\text{Sig}_B || B$; verify with $\text{Verify}_{pk_B}(\text{Sig}_B, B)$. Abort on failure.
9. **Alice:** compute shared secret $S = \text{X25519}(a, B)$ (mathematically $S = g^{ab}$).
10. **Bob:** compute shared secret $S = \text{X25519}(b, A)$ (same $S = g^{ab}$).
11. **Both:** derive symmetric AES key $K = \text{HKDF-SHA256}(\text{IKM} = S, \text{length} = 32, \text{info} = \text{"handshake data"})$.
12. **Alice:** encrypt payload with AES-GCM (nonce 12B, tag 16B) using K and send $\text{nonce} || \text{tag} || \text{ciphertext}$.
13. **Bob:** decrypt and verify AES-GCM tag with K , then process plaintext.

Key Delineation

- **Ephemeral keys (session):** X25519 keys (private values a, b and public points A, B).
 - Purpose: provide a fresh Diffie-Hellman shared secret per session and enable forward secrecy.
 - Property: ephemeral; if a key is compromised later, it does not reveal past session keys if a, b are unrecovered.
- **Long-term identity keys:** RSA-2048 identity keypairs ($sk_A / pk_A, sk_B / pk_B$).
 - Purpose: authenticate the origin of an ephemeral X25519 public value by signing it.
 - Property: persistent identity binding; used only for signatures (not for deriving session secrets).

Why sign the X25519 public bytes before transmission

Problem if unsigned: an active attacker M can perform a man-in-the-middle (MitM) by substituting ephemeral public values. Without authentication:

- Alice sends $A = g^a$, Bob sends $B = g^b$.
- Attacker M intercepts and substitutes $A' = g^{a'}$ to Bob and $B' = g^{b'}$ to Alice.
- Alice computes $S_A = g^{ab'}$, Bob computes $S_B = g^{a'b}$, while M knows both $g^{ab'}$ and $g^{a'b}$ and can decrypt/relay messages.

Thus unauthenticated Diffie-Hellman is vulnerable to MitM and Unknown-Key-Share (UKS) attacks.

Authentication by signing fixes this: each ephemeral public key is accompanied by a signature under the sender's long-term private key. Acceptance requires

$$\text{Verify}(pk_B, \text{Sig}_B, B) = \text{True}$$

which implies (under the EUF-CMA security of RSA-PSS) that the sender holds sk_B and intentionally produced B . An attacker who tries to substitute B' must provide a matching signature Sig' such that

$$\text{Verify}(pk_B, \text{Sig}', B') = \text{True},$$

but forging such a pair without sk_B is computationally infeasible. Therefore substitution attacks are prevented.

Mathematically, the honest shared secret is

$$S = g^{ab}$$

and both sides only compute S from ephemeral values that have been proven (by signatures) to originate from the claimed identities. This enforces authentication of the Diffie-Hellman exchange and prevents MitM and Unknown-Key-Share.

Security properties achieved

- Authentication of peers (via RSA signatures over ephemeral X25519 pubs).
- Forward secrecy (ephemeral X25519 keys provide Diffie-Hellman secrecy independent of long-term key compromise).
- Resistance to active MitM and UKS attacks (signature binding prevents substitution and impersonation).

Security properties achieved

- Authenticated peers
- No harm from long-term key compromise
- Resistance to MitM and Unknown-Key-Share attacks

3. Network Verification

Network packet analysis confirms that the cryptographic boundary remains intact and that plaintext never escapes the Docker bridged network (br-crypto).

✓ Outcome

This project successfully demonstrates the manual implementation of a modern, hybrid secure protocol architecture, providing a blueprint for secure peer-to-peer communication.

Week 1: Proof of Confidentiality

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.18.0.3	172.18.0.2	TCP	74	57118 → 65432 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK PERM TSval=2174685701 TSecr=0 WS=128
2	0.000051	172.18.0.2	172.18.0.3	TCP	74	65432 → 57118 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK PERM TSval=221600820 TSecr=2174685701 WS=128
3	0.000065	172.18.0.3	172.18.0.2	TCP	66	57118 → 65432 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2174685702 TSecr=221600820
4	0.000136	172.18.0.3	172.18.0.2	TCP	134	57118 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=68 TSval=2174685702 TSecr=221600820
5	0.000143	172.18.0.2	172.18.0.3	TCP	66	65432 → 57118 [ACK] Seq=1 Ack=69 Win=65152 Len=0 TSval=221600821 TSecr=2174685702
6	0.000167	172.18.0.3	172.18.0.2	TCP	66	57118 → 65432 [FIN, ACK] Seq=69 Ack=1 Win=64256 Len=0 TSval=2174685702 TSecr=221600821
7	0.001552	172.18.0.2	172.18.0.3	TCP	66	65432 → 57118 [FIN, ACK] Seq=1 Ack=70 Win=65152 Len=0 TSval=221600822 TSecr=2174685702
8	0.001571	172.18.0.3	172.18.0.2	TCP	66	57118 → 65432 [ACK] Seq=70 Ack=2 Win=64256 Len=0 TSval=2174685703 TSecr=221600822

Frame 4: 134 bytes on wire (1072 bits), 134 bytes captured (1072 bits)

Ethernet II, Src: 0e:4a:5f:ea:0a:86 (0e:4a:5f:ea:0a:86), Dst: 6e:6a:40:c1:d8:ff (6e:6a:40:c1:d8:ff)

Internet Protocol Version 4, Src: 172.18.0.3, Dst: 172.18.0.2

Transmission Control Protocol, Src Port: 57118, Dst Port: 65432, Seq: 1, Ack: 1, Len: 68 (68 bytes)

Data (68 bytes)

Data: a1710aa37666116f4c0dc135ef2680c8d951975613357c4e5e98cd9f083651dee811f65d7a0cf3 [Length: 68]

0000 6e 6a 40 c1 d8 ff 0e 4a 5f ea 0a 86 08 00 45 00 nj@...J _...E-
0010 00 78 d6 61 40 00 00 06 0b f5 ac 12 00 03 ac 12 -x a@ _...
0020 00 02 df 1e ff 98 5c f7 2a c5 bf 9a c0 46 80 18 \ *...F..
0030 01 f6 58 94 00 00 01 01 08 0a 81 9f 12 06 0d 35 -X.....S
0040 5c 34 a1 71 0a a3 76 66 11 6f 4c 0d c1 35 ef 26 \4q...vf .oL..5..
0050 80 c8 d9 51 97 56 13 35 7c 4e 5e 98 cd 9f 08 36 ...Q.V:5 |N*....6
0060 51 de e8 11 f6 5d 7a 0c f3 36 4e ee e2 9c 4b Q....|z .6N....K
0070 09 04 c9 76 c9 ad 55 f8 c2 c0 50 01 62 94 a9 40U...P.b...e
0080 aa e4 8e 1f 1e 2f

As seen in the above capture, Frame 4 shows a total of 134 bytes on the wire. This includes the 68-byte "CLASSIFIED" data payload. The "cryptographic expansion" resulting in 134 bytes is accounted for by the 12-byte nonce, 16-byte GCM tag, and 38 bytes of Ethernet/IP/TCP overhead. The hex dump (a1 71 0a a3 76 66...) shows that the data is indistinguishable from random noise, proving the AES-GCM encryption is effective.

Week 1: Proof of Integrity

```
# python server.py
[*] Bob (Server) listening on 0.0.0.0:65432...
[*] Connection established from ('172.18.0.3', 50158)
[*] Received 68 bytes over the wire.
[-] Decryption failed. Invalid authentication tag. Possible tampering detected.
#
```

The above capture shows the result of a controlled bit-flip in the ciphertext. The server output: Decryption failed. Invalid authentication tag. Possible tampering detected. confirms that the GCM mode successfully identified the modification and halted processing, fulfilling the integrity requirement.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.18.0.3	172.18.0.2	TCP	74	37086 → 65432 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2259468100 TSecr=0 WS=128
2	0.000030	172.18.0.2	172.18.0.3	TCP	74	65432 → 37086 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=306383219 TSecr=2259468100 WS=128
3	0.000043	172.18.0.3	172.18.0.2	TCP	66	37086 → 65432 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2259468100 TSecr=306383219
4	0.003932	172.18.0.3	172.18.0.2	TCP	98	37086 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=32 TSval=2259468104 TSecr=306383219
5	0.003943	172.18.0.2	172.18.0.3	TCP	66	65432 → 37086 [ACK] Seq=1 Ack=33 Win=65152 Len=0 TSval=306383223 TSecr=2259468104
6	0.005147	172.18.0.2	172.18.0.3	TCP	98	65432 → 37086 [PSH, ACK] Seq=1 Ack=33 Win=65152 Len=32 TSval=306383224 TSecr=2259468104
7	0.005167	172.18.0.3	172.18.0.2	TCP	66	37086 → 65432 [ACK] Seq=33 Ack=33 Win=64256 Len=0 TSval=2259468105 TSecr=306383224
8	0.005706	172.18.0.3	172.18.0.2	TCP	134	37086 → 65432 [PSH, ACK] Seq=33 Ack=33 Win=64256 Len=68 TSval=2259468106 TSecr=306383224
9	0.005749	172.18.0.3	172.18.0.2	TCP	66	37086 → 65432 [FIN, ACK] Seq=101 Ack=33 Win=64256 Len=0 TSval=2259468106 TSecr=306383224
10	0.006297	172.18.0.2	172.18.0.3	TCP	66	65432 → 37086 [FIN, ACK] Seq=33 Ack=102 Win=65152 Len=0 TSval=306383225 TSecr=2259468106
11	0.006316	172.18.0.3	172.18.0.2	TCP	66	37086 → 65432 [ACK] Seq=102 Ack=34 Win=64256 Len=0 TSval=2259468106 TSecr=306383225

▶ Frame 4: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) ▶ Ethernet II, Src: f6:51:f8:bc:82:33 (f6:51:f8:bc:82:33), Dst: 32:5c:2d:51:44:f4 (32:5c:2d:51:44:f4) ▶ Internet Protocol Version 4, Src: 172.18.0.3, Dst: 172.18.0.2 ▶ Transmission Control Protocol, Src Port: 37086, Dst Port: 65432, Seq: 1, Ack: 1, Len: 32 ▶ Data (32 bytes) Data: a8891583c7c38ec57aa1278e633041d142b8328547809befdbdf782adc888146 [Length: 32]	<pre> 0000 32 5c 2d 51 44 f4 f6 51 f8 bc 82 33 08 00 45 00 2\-\00-\0 ...3-E- 0010 00 54 c5 72 40 00 40 06 1d 08 ac 12 00 03 ac 12 x t @ @ 0020 00 02 90 de ff 98 60 12 d4 1c fb a3 c5 bb 08 18 0030 01 f6 58 70 00 00 01 01 08 0a 86 ac bf 48 12 43 -Xp.....H-C 0040 09 73 08 09 15 83 c7 c3 8e c5 7a a1 27 06 03 30 -S.....z'-C 0050 41 d1 42 b8 32 85 47 80 9b ef db df 78 2a dc 88 A B : 2 Gx- 0060 81 46 -F </pre>
--	--

The Wireshark capture above highlights the ephemeral ECDH "bootstrapping" process. Two 32-byte X25519 public keys are clearly visible (the first shown in screenshot) in the TCP stream, exchanged in plaintext to negotiate a shared secret. These points serve as the mathematical foundation that will create a secure session (even though they are exposed on the wire). This implementation successfully transitions the pipeline from a hardcoded pre-shared key to a dynamically negotiated key exchange.

Week 2: Proof of Payload

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.18.0.3	172.18.0.2	TCP	74	37086 → 65432 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2259468100 TSecr=0 WS=128
2	0.000030	172.18.0.2	172.18.0.3	TCP	74	65432 → 37086 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=306383219 TSecr=2259468100 WS=128
3	0.000043	172.18.0.3	172.18.0.2	TCP	66	37086 → 65432 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2259468100 TSecr=306383219
4	0.003932	172.18.0.3	172.18.0.2	TCP	98	37086 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=32 TSval=2259468104 TSecr=306383219
5	0.003943	172.18.0.2	172.18.0.3	TCP	66	65432 → 37086 [ACK] Seq=1 Ack=33 Win=65152 Len=0 TSval=306383223 TSecr=2259468104
6	0.005147	172.18.0.2	172.18.0.3	TCP	98	65432 → 37086 [PSH, ACK] Seq=1 Ack=33 Win=65152 Len=32 TSval=306383224 TSecr=2259468104
7	0.005167	172.18.0.3	172.18.0.2	TCP	66	37086 → 65432 [ACK] Seq=33 Ack=33 Win=64256 Len=0 TSval=2259468105 TSecr=306383224
8	0.005706	172.18.0.3	172.18.0.2	TCP	134	37086 → 65432 [PSH, ACK] Seq=33 Ack=33 Win=64256 Len=68 TSval=2259468106 TSecr=306383224
9	0.005749	172.18.0.3	172.18.0.2	TCP	66	37086 → 65432 [FIN, ACK] Seq=101 Ack=33 Win=64256 Len=0 TSval=2259468106 TSecr=306383224
10	0.006297	172.18.0.2	172.18.0.3	TCP	66	65432 → 37086 [FIN, ACK] Seq=33 Ack=102 Win=65152 Len=0 TSval=306383225 TSecr=2259468106
11	0.006316	172.18.0.3	172.18.0.2	TCP	66	37086 → 65432 [ACK] Seq=102 Ack=34 Win=64256 Len=0 TSval=2259468106 TSecr=306383225

▶ Frame 8: 134 bytes on wire (1072 bits), 134 bytes captured (1072 bits) ▶ Ethernet II, Src: f6:51:f8:bc:82:33 (f6:51:f8:bc:82:33), Dst: 32:5c:2d:51:44:f4 (32:5c:2d:51:44:f4) ▶ Internet Protocol Version 4, Src: 172.18.0.3, Dst: 172.18.0.2 ▶ Transmission Control Protocol, Src Port: 37086, Dst Port: 65432, Seq: 33, Ack: 33, Len: 68 ▶ Data (68 bytes) Data: 52f52791945876f2af7359201f83976b8e8c3753128de2d2ea3d8966f825a0887aafee8dc206 [Length: 68]	<pre> 0000 32 5c 2d 51 44 f4 f6 51 f8 bc 82 33 08 00 45 00 2\-\00-\0 ...3-E- 0010 00 78 c5 74 40 00 40 06 1c e2 ac 12 00 03 ac 12 x t @ @ 0020 00 02 90 de ff 98 60 12 d4 3c fb a3 c5 db 08 18 0030 01 f6 58 94 00 00 01 01 08 0a 86 ac bf 4a 12 43 -Xp.....J-C 0040 09 73 08 09 15 83 c7 c3 8e c5 7a a1 27 06 03 30 -S.....z'-C 0050 97 6b 8e 8c 37 33 12 0d e2 d2 ea 3d 89 66 f8 25 -K-7S.....f-% 0060 a0 88 7a af ee 0d c2 06 54 d5 88 30 7e 93 72 59 -T-0--rY 0070 a6 74 5d e2 e8 67 83 0b 04 7c 98 a5 96 13 d6 75 -t\...g...-u 0080 da d5 04 b6 5f 8e </pre>
--	---

Despite the previous public keys being visible, the session is now fully encrypted.

Following the public key exchange, the transmission shows the fully secured data payload. Even though the public keys were visible, the resulting AES-GCM ciphertext remains indistinguishable from random noise. This confirms that the shared secret was correctly derived via HKDF using SHA-256 to produce a unique 32-byte AES key, maintaining confidentiality.

4. Defensive Implementation and Code Audit Log

Defensive coding is the bedrock of cryptographic software, prioritizing secure entropy and fail-safe defaults to ensure the system does not leak information.

Entropy Generation (`client.py` • line 20)

Within the `encrypt_payload` function, the implementation utilizes `os.urandom(12)` to generate a unique nonce. This satisfies the NIST requirement for secure randomness by utilizing the operating system's CSPRNG, ensuring that no two sessions share a (Key, Nonce) pair.

Defense against Timing Attacks

Although the code in this project does not use HMAC (not in code requirements), an authentication phase in general should use constant-time comparisons. This prevents attackers from using timing leaks to measure the response times and discover the RSA signatures. For more details, see section [Week 3 Timing Attacks Side-Channel Defense and HMAC](#).

Fail-Safe Error Handling (`server.py` • lines 139-140, 147-148)

The server contains a try/except block to catch `InvalidTag` and `InvalidSignature` exceptions. If the RSA signature verification fails or the GCM tag is incorrect, the system executes a fail-safe default: it logs the failure and drops the connection immediately. This prevents recovery attacks and ensures that the system provides no "padding oracle" feedback to an attacker.

5. AI Analysis vs. NIST/RFC Standards Validation

In security engineering, Generative AI can provide a quick, amplified explanation of concepts. However, true quality assurance comes in rigorous human verification against official standards.

Week 1: AES-GCM and NIST SP 800-38D

The AI's created an explanation of Galois field multiplication and nonce constraints that is accurate to NIST SP 800-38D standard. The AI correctly identified that AES-GCM uses a universal hash function (GHASH) defined over the binary Galois field $GF(s^{128})$. It also correctly represented GHASH as the following polynomial evaluation:

$$\sum X_i \cdot H^i$$

This matches NIST's specification for generating the has subkey H from a block of zeros and the subsequent block multiplication operation. The AI did not hallucinate the "catastrophic" nature of nonce reuse – Something that NIST explicitly warns about when making IVs. The AI also provided a technically sound summary of the algebraic properties NIST describes as GHASH's foundation.

Week 2: X25519 vs. NIST P-256 (RFC 7748)

The AI's description of the difference between short Weierstrass (NIST P-256) and Montgomery (X25519) curves is technically accurate, showing that they differ in mathematical form:

$$y^2 = x^3 + ax + b$$

vs

$$y^2 = x^3 + Ax^2 + x$$

The AI also highlighted the (accurate) difference in design philosophy and implementation safety. Weierstrass curves require rigorous point validation and complex formulas that increase the risk of timing and invalid curve attacks. While X25519 uses the Montgomery ladder, which is naturally $O(1)$ and significantly more misuse-resistant. These facts were highlighted by the AI and are accurate to the RFC 7748 standards. The historical concerns around these NIST P-256 weaknesses are valid, mainly due to unexplained curve constants by the creators. This distrust, along with an instance where an NSA influenced algorithm (Dual_EC_DRBG) contained a backdoor, resulted in the development of X25519 and its inclusion in RFC 7748. X25519 is more trustworthy due to its explained constants, simplicity, and overall transparency.

Week 3: Timing Attacks Side-Channel Defense and HMAC

A timing vulnerability is a side-channel attack where secret data (such as a MAC, API token, or digital signature) can be stolen by measuring the time it takes a system to process a request. The culprit for this attack is using standard string comparisons (like `==`), which cause the program to exit early (because of a mismatch) but that early exit gives off information on how much of the string was checked (letting the attacker know how many digits they got right). Over many samples, an attacker can statistically filter out network noise to recover a secret one character at a time.

To fix this, you replace the simple character comparison (`==`) with Python's `hmac.compare_digest(a, b)`. Unlike naive comparisons, `hmac.compare_digest()` is designed to operate in $O(1)$ constant time, meaning the total execution time remains the same regardless of what string is thrown into it. No early exit happens -> No timing info is given away.

Appendix

AI Accreditation

NotebookLM: [Secure Hybrid Cryptographic Pipeline Implementation](#)

Copilot in VS Code: [2. Protocol Architecture and Authenticated Handshake Flow](#)

ChatGPT: [5. AI Analysis vs. NIST/RFC Standards Validation](#) • [Chat log](#)

Full Code Listings

`client.py`

```
import socket
import os
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives.asymmetric import x25519
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.serialization import Encoding, PublicFormat
from cryptography.exceptions import InvalidSignature

def load_keys():
    """Loads Alice's private identity key and Bob's public identity key."""
    with open("alice_private.pem", "rb") as f:
        alice_priv = serialization.load_pem_private_key(f.read(), password=None)
    with open("bob_public.pem", "rb") as f:
        bob_pub = serialization.load_pem_public_key(f.read())
    return alice_priv, bob_pub

def encrypt_payload(plaintext: bytes, key: bytes) -> tuple[bytes, bytes, bytes]:
    nonce = os.urandom(12)
    cipher = Cipher(algorithms.AES(key), modes.GCM(nonce))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext) + encryptor.finalize()
    tag = encryptor.tag
    return (nonce, ciphertext, tag)

def recv_exact(sock: socket.socket, size: int) -> bytes:
    data = b""
    while len(data) < size:
        chunk = sock.recv(size - len(data))
        if not chunk:
            raise ConnectionError("Connection closed before receiving expected data.")
        data += chunk
    return data

def derive_aes_key(shared_secret: bytes) -> bytes:
    """
    DONE: Implement HKDF to derive a 32-byte AES key.
    Must match the Server's HKDF parameters exactly.
    """
    hkdf = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=None,
        info=b"handshake data",
    )
    return hkdf.derive(shared_secret)
```

```

def start_client():
    host, port = 'bob_node', 65432
    message = b"CLASSIFIED: The Eagle flies at midnight."
    alice_identity_priv, bob_identity_pub = load_keys()

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        try:
            s.connect((host, port))
            print(f"[*] Connected to Bob at {host}:{port}")

            # --- PHASE 1: AUTHENTICATED ECDH HANDSHAKE ---
            # DONE: 1. Generate Alice's ephemeral X25519PrivateKey
            alice_private_key = x25519.X25519PrivateKey.generate()

            # DONE: 2. Extract Alice's X25519 public bytes
            alice_public_bytes = alice_private_key.public_key().public_bytes(
                encoding=Encoding.Raw,
                format=PublicFormat.Raw,
            )

            # DONE: 3. Use alice_identity_priv to SIGN the X25519 public bytes using padding.PSS and
            hashes.SHA256()
            signature = alice_identity_priv.sign(
                alice_public_bytes,
                padding.PSS(
                    mgf=padding.MGF1(hashes.SHA256()),
                    salt_length=padding.PSS.MAX_LENGTH
                ),
                hashes.SHA256()
            )

            # DONE: 4. Send Alice's signature (256 bytes) and Alice's X25519 public key (32 bytes) to
            Bob.
            s.sendall(signature + alice_public_bytes)

            # DONE: 5. Receive Bob's data (Format: 256-byte signature + 32-byte X25519 public key)
            bob_data = recv_exact(s, 288)
            bob_signature = bob_data[:256]
            bob_public_bytes = bob_data[256:]

            # DONE: 6. Use bob_identity_pub to VERIFY Bob's signature against his X25519 public bytes.
            bob_identity_pub.verify(
                bob_signature,
                bob_public_bytes,
                padding.PSS(
                    mgf=padding.MGF1(hashes.SHA256()),
                    salt_length=padding.PSS.MAX_LENGTH
                ),
                hashes.SHA256()
            )

            # DONE: 7. Compute the shared_secret using the verified X25519 points.
            shared_secret = alice_private_key.exchange(
                x25519.X25519PublicKey.from_public_bytes(bob_public_bytes)
            )

            # --- PHASE 2: KEY DERIVATION ---
            # DONE: Derive the AES key using HKDF (Same as Week 2)
            aes_key = derive_aes_key(shared_secret)

            # --- PHASE 3: SECURE PAYLOAD ---

```

```

        # DONE: Encrypt and send the AES-GCM payload (Same as Week 1 & 2)
        nonce, ciphertext, tag = encrypt_payload(message, aes_key)
        s.sendall(nonce + tag + ciphertext)

    except ConnectionRefusedError:
        print("[!] Connection refused. Is Bob's server running?")
    except ConnectionError as e:
        print(f"[!] Handshake failed: {e}")
    except InvalidSignature:
        print("[!] Handshake failed: Invalid signature from Bob. Authentication failed.")
    except ValueError as e:
        print(f"[!] Invalid peer public key or cryptographic parameters: {e}")

if __name__ == "__main__":
    start_client()

```

server.py

```

import socket
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives.asymmetric import x25519
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.exceptions import InvalidTag, InvalidSignature
from cryptography.hazmat.primitives.serialization import Encoding, PublicFormat

def load_keys():
    """Loads Bob's private identity key and Alice's public identity key."""
    with open("bob_private.pem", "rb") as f:
        bob_priv = serialization.load_pem_private_key(f.read(), password=None)
    with open("alice_public.pem", "rb") as f:
        alice_pub = serialization.load_pem_public_key(f.read())
    return bob_priv, alice_pub

def decrypt_payload(nonce: bytes, ciphertext: bytes, tag: bytes, key: bytes) -> bytes:
    cipher = Cipher(algorithms.AES(key), modes.GCM(nonce, tag))
    decryptor = cipher.decryptor()
    plaintext = decryptor.update(ciphertext) + decryptor.finalize()
    return plaintext

def rcv_exact(sock: socket.socket, size: int) -> bytes:
    data = b""
    while len(data) < size:
        chunk = sock.recv(size - len(data))
        if not chunk:
            raise ConnectionError("Connection closed before receiving expected data.")
        data += chunk
    return data

def derive_aes_key(shared_secret: bytes) -> bytes:
    """
    DONE: Implement HKDF to derive a 32-byte AES key.
    Use SHA256 as the algorithm, length=32, salt=None, info=b'handshake data'.
    """
    hkdf = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=None,
        info=b"handshake data",
    )

```

```

    return hkdf.derive(shared_secret)

def start_server():
    host, port = '0.0.0.0', 65432
    bob_identity_priv, alice_identity_pub = load_keys()

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((host, port))
        s.listen()
        print(f"[*] Bob (Server) listening on {host}:{port}...")

    conn, addr = s.accept()
    with conn:
        print(f"[*] Connection established from {addr}")

        try:
            # --- PHASE 1: AUTHENTICATED ECDH HANDSHAKE ---
            # DONE: 1. Generate Bob's ephemeral X25519PrivateKey
            bob_private_key = x25519.X25519PrivateKey.generate()

            # DONE: 2. Extract Bob's X25519 public bytes
            bob_public_bytes = bob_private_key.public_key().public_bytes(
                encoding=Encoding.Raw,
                format=PublicFormat.Raw,
            )

            # DONE: 3. Use bob_identity_priv to SIGN the X25519 public bytes using padding.PSS and
            hashes.SHA256()
            signature = bob_identity_priv.sign(
                bob_public_bytes,
                padding.PSS(
                    mgf=padding.MGF1(hashes.SHA256()),
                    salt_length=padding.PSS.MAX_LENGTH
                ),
                hashes.SHA256()
            )

            # DONE: 4. Receive Alice's data (Format: 256-byte signature + 32-byte X25519 public key)
            alice_data = recv_exact(conn, 288)
            alice_signature = alice_data[:256]
            alice_public_bytes = alice_data[256:]

            # DONE: 5. Use alice_identity_pub to VERIFY Alice's signature against her X25519 public
            bytes.
            # (If verification fails, cryptography will raise an InvalidSignature exception.
            Let it crash the connection.)
            alice_identity_pub.verify(
                alice_signature,
                alice_public_bytes,
                padding.PSS(
                    mgf=padding.MGF1(hashes.SHA256()),
                    salt_length=padding.PSS.MAX_LENGTH
                ),
                hashes.SHA256()
            )

            # DONE: 6. Send Bob's signature and Bob's X25519 public bytes to Alice.
            conn.sendall(signature + bob_public_bytes)

            # DONE: 7. Compute the shared_secret using the verified X25519 points.
            shared_secret = bob_private_key.exchange(

```

```

        x25519.X25519PublicKey.from_public_bytes(alice_public_bytes)
    )

# --- PHASE 2: KEY DERIVATION ---
# DONE: Derive the AES key using HKDF (Same as Week 2)
aes_key = derive_aes_key(shared_secret)

# --- PHASE 3: RECEIVE PAYLOAD ---
# DONE: Receive and decrypt the AES-GCM payload (Same as Week 1 & 2)
chunks = []
while True:
    chunk = conn.recv(4096)
    if not chunk:
        break
    chunks.append(chunk)
data = b"".join(chunks)

if not data:
    print("[~] No payload data received.")
    return

if data:
    # DONE: Extract nonce, tag, and ciphertext.
    # DONE: Call your Week 1 decrypt_payload() using the NEW derived key.
    if len(data) < 28:
        print("[~] Payload too short to contain nonce + tag + ciphertext.")
        return

    nonce = data[:12]
    tag = data[12:28]
    ciphertext = data[28:]

    try:
        plaintext = decrypt_payload(nonce, ciphertext, tag, aes_key)
        print("[*] Payload decrypted successfully.")
        print(f"[*] Decrypted plaintext: {plaintext.decode()}")
    except InvalidTag:
        print("[~] Decryption failed. Invalid authentication tag. Possible tampering
detected.")

    except UnicodeDecodeError:
        print("[~] Decryption succeeded, but plaintext is not valid UTF-8.")
except ConnectionError as e:
    print(f"[~] Handshake failed: {e}")
except ValueError as e:
    print(f"[~] Handshake failed: {e}")
except InvalidSignature:
    print("[~] Handshake failed: Invalid signature. Authentication failed.")

if __name__ == "__main__":
    start_server()

```